

Центр разработки радиочастотных систем

Вариант 2

Задача:

1. Реализовать на Matlab, C, C++ или Python КИХ фильтр с произвольного выбранными порядком и характеристикой, где сигнал и коэффициенты фильтра представлены комплексными числами с фиксированной точкой. Проверить, что фильтр работает как ожидается (описать выбранный способ проверки и результат)
2. Нарисовать одну из возможных схем реализации фильтра в RTL (на ПЛИС или в ASIC). Указать использование регистров и элементарных математических операций: сложение, инверсия знака, умножение в действительных числах и т.п. Оценить требуемое число элементарных операций каждого вида в зависимости от порядка фильтра.
3. Предложить идеи, как можно уменьшить число операций, если известно, что отсчеты сигнала поступают на вход фильтра ровно 1 раз в 4 такта. Нарисовать измененную схему реализации. Оценить изменение количества требуемых операций

Аннотация

В работе использовался КИХ-фильтр высоких частот с порядком $N = 19$ с окном Хэмминга. Рассматривались фильтры с аналогичными характеристиками с окнами Кайзера для сравнения. Частота дискретизации была взята 1000. А диапазон нормализован на частоту Найквиста. Реализации и визуализации происходила на языке Python.

Ход работы

Код представлен файле python.

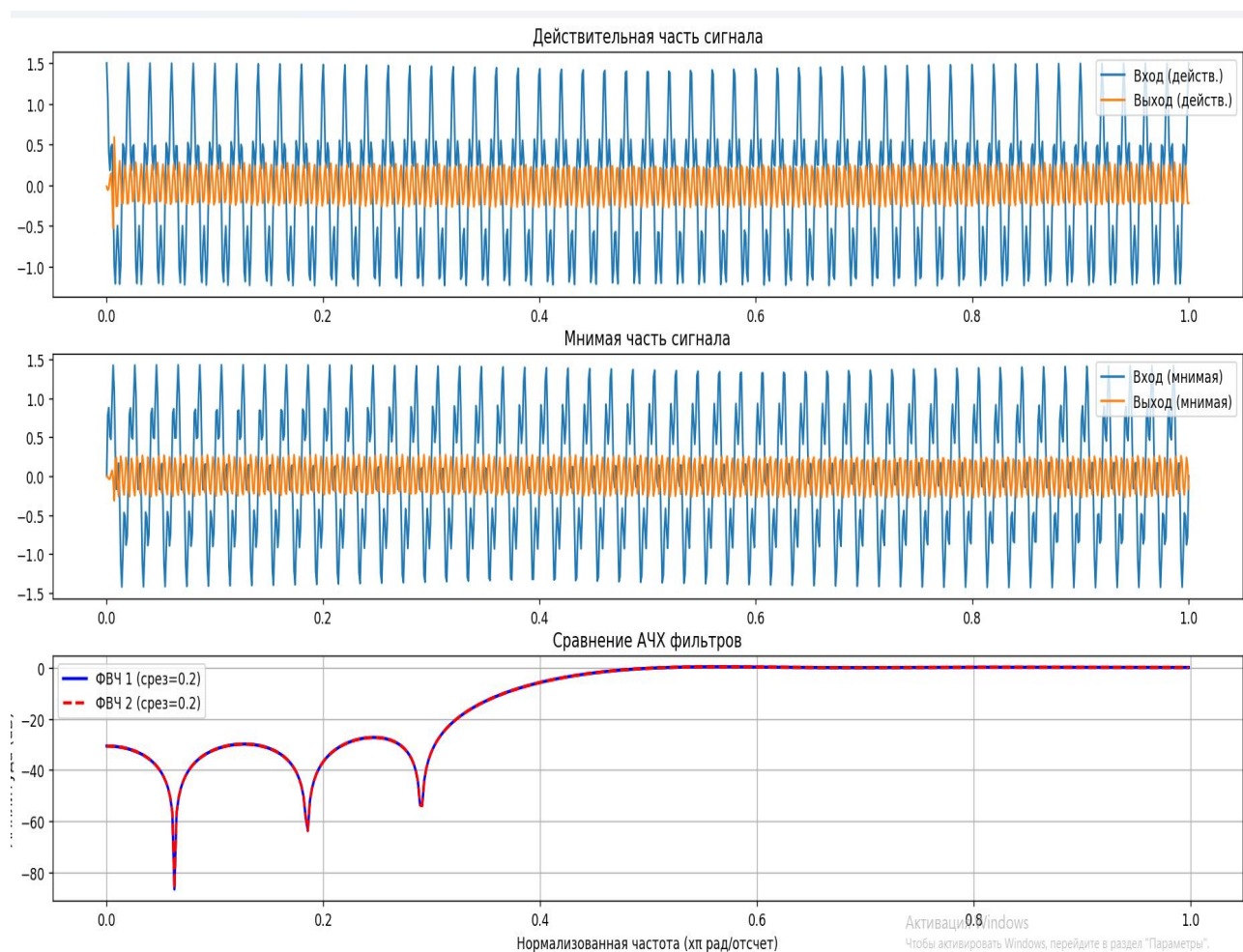


Рисунок 1 – компоненты входного и выходного сигнала, а также АЧХ КИХ-фильтров. ФВЧ 1 – с окном Хэмминга, ФВЧ 2 – с окном Кайзера ($\beta = 4.0$)

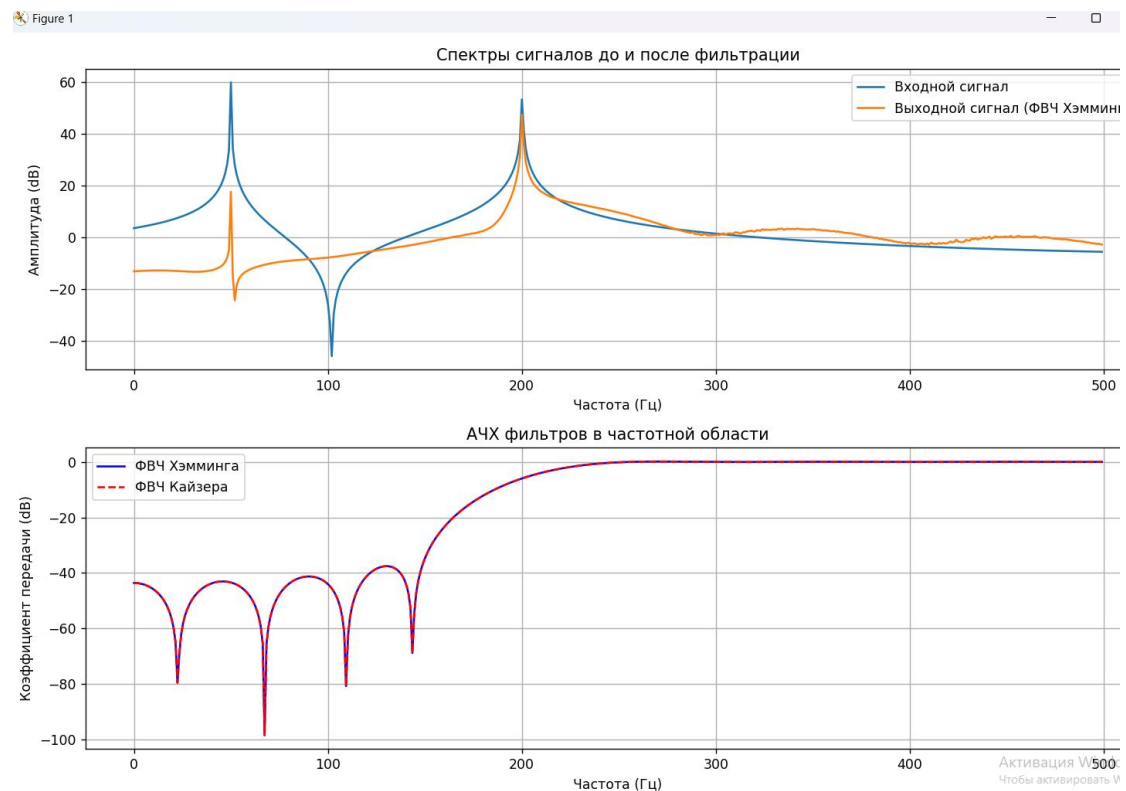


Рисунок 2 – спектр входного и выходного сигнала, а также АЧХ обоих КИХ-фильтров.

На рисунке 2 видно как пик на частоте 50 МГц с 60 дБ понизился примерно до 30 дБ в соответствии с АЧХ фильтра 1. В то же время, пик на 200 МГц, попадающий в полосу пропускания ФВЧ 1, понизился примерно на 5 дБ, что в сравнении с предыдущим пиком немного. Из-за этого пик на 200 МГц превосходит пик на 50 МГц на выходе из ФВЧ с окном Хэмминга.

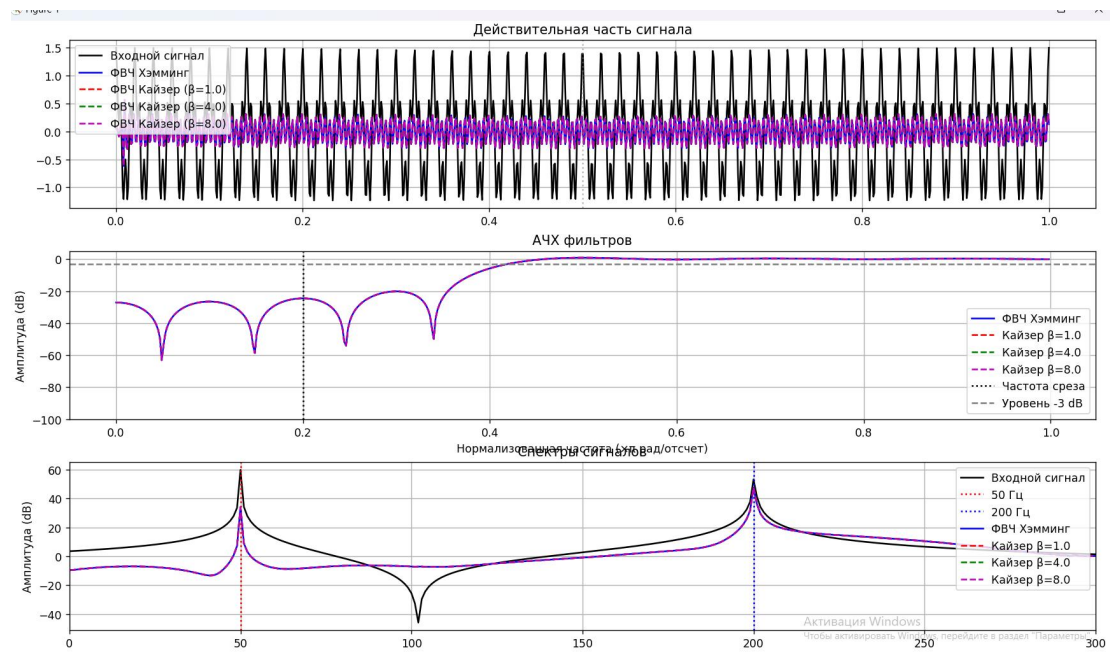


Рисунок 3 – спектр входного и выходного сигнала, а также АЧХ КИХ-фильтров с окнами Кайзеора для разных β .

Пример реализации на ПЛИС

В современных ПЛИС разрядность коэффициентов может быть выбрана любой, но разумные цифры лежат в пределах от 16 до 27 битов. Для высоких порядков фильтра часто требуется обеспечить большой динамический диапазон разрядной сетки, но если этого не удастся сделать, рано или поздно начинают проявляться ошибки квантования. (Статья на Хабр "*Цифровая фильтрация на ПЛИС. ч.2*")

СХЕМА

Вход -> Демультимплексор (1:4) -> 4 подфильтра -> Коммутатор -> Выход

Операция	Кол-во	Примечания
Регистры	38	19*2
Умножители	76	19*4
Сумматоры	38	2*19
Инверсия	38	Для вычитания

КОД

```
module FIR_Filter (  
  
    input clk, rst,  
  
    input signed [7:0] real_in, imag_in, // 8-бит вход (фикс. точка Q8.8)  
  
    output signed [7:0] real_out, imag_out  
  
);  
  
    // Коэффициенты фильтра (пример для N=19)  
  
    wire signed [7:0] h_real[0:18]; // Реальная часть коэффициентов  
  
    wire signed [7:0] h_imag[0:18]; // Мнимая часть (в вашем коде =0)  
  
  
    // Регистры задержки (цепочка сдвига)  
  
    reg signed [7:0] real_delay[0:18];  
  
    reg signed [7:0] imag_delay[0:18];  
  
  
    // Арифметические блоки  
  
    always @(posedge clk) begin  
  
        if (rst) begin  
  
            // Сброс регистров
```

```
end else begin
```

```
// Сдвиг данных в линии задержки
```

```
for (int i=18; i>0; i--) begin
```

```
    real_delay[i] <= real_delay[i-1];
```

```
    imag_delay[i] <= imag_delay[i-1];
```

```
end
```

```
real_delay[0] <= real_in;
```

```
imag_delay[0] <= imag_in;
```

```
end
```

```
end
```

```
// Вычислительное ядро
```

```
wire signed [19:0] real_acc, imag_acc; // Накопители
```

```
always_comb begin
```

```
    real_acc = 0;
```

```
    imag_acc = 0;
```

```
    for (int i=0; i<19; i++) begin
```

```
        // Комплексное умножение:  $(a+jb)*(c+jd) = (ac-bd) + j(ad+bc)$ 
```

```
    real_acc += real_delay[i]*h_real[i] - imag_delay[i]*h_imag[i];  
  
    imag_acc += real_delay[i]*h_imag[i] + imag_delay[i]*h_real[i];  
  
end  
  
end  
  
assign real_out = real_acc[19:8]; // Обратное масштабирование  
  
assign imag_out = imag_acc[19:8];  
  
endmodule
```